

Q.No:10

Question description

You are given a list consisting of n integers. Your task is to remove elements from the list at given positions, and report the removed elements.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x_i \leq 10^9$
- $1 \leq p_i \leq n-i+1$

Input

The first input line has an integer n : the initial size of the list. During the process, the elements are numbered $1, 2, \dots, k$ where k is the current size of the list.

The second line has n integers $x_1, x_2, \dots, x_n$: the contents of the list.

The last line has n integers $p_1, p_2, \dots, p_n$: the positions of the elements to be removed.

Output

Print the elements in the order they are removed.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
2 6 1 4 2
3 1 3 1 1
```

EXPECTED OUTPUT

```
1 2 2 6 4
```

Test Case 2

INPUT (STDIN)

```
6
7 2 6 1 4 2
11 3 1 3 1 1
```

EXPECTED OUTPUT

```
2 6 7 4 2 1
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void build(int k, int
l, int r)
```

Code:

```
#include <stdio.h>

#define MAX 200005

int tree[MAX];
int a[MAX];

void build(int k, int l, int r)
{
```

```
if(l == r)
{
    tree[k] = 1;
    return;
}

int mid = (l + r) / 2;

build(2 * k, l, mid);
build(2 * k + 1, mid + 1, r);

tree[k] = tree[2 * k] + tree[2 * k + 1];
}

int removeElement(int k, int l, int r, int pos)
{
    if(l == r)
    {
        tree[k] = 0;
        return l;
    }

    int mid = (l + r) / 2;
    int index;

    if(tree[2 * k] >= pos)
    {
        index = removeElement(2 * k, l, mid, pos);
    }
    else
    {
        index = removeElement(2 * k + 1, mid + 1, r,
                               pos - tree[2 * k]);
    }

    tree[k] = tree[2 * k] + tree[2 * k + 1];

    return index;
}

int main()
{
    int n;
    int p[MAX];
    int i;

    scanf("%d", &n);
    for(i = 1; i <= n; i++)
    {
        scanf("%d", &a[i]);
    }
}
```

```
for(i = 1; i <= n; i++)  
{  
    scanf("%d", &p[i]);  
}  
build(1, 1, n);  
  
printf("Output:\n");  
for(i = 1; i <= n; i++)  
{  
    int index;  
  
    index = removeElement(1, 1, n, p[i]);  
  
    printf("%d", a[index]);  
  
    if(i < n)  
        printf(" ");  
}  
  
printf("\n");  
  
return 0;  
}
```

Output:

Test_case 1:

```
5  
2 6 1 4 2  
3 1 3 1 1  
Output:  
1 2 2 6 4
```

Test_case 2:

```
6  
7 2 6 1 4 2  
11 3 1 3 1 1  
Output:  
2 6 7 4 2 1
```